5/29/2026

Imran Nazeer

DHC-2094 (Cybersecurity Intern)

https://github.com/ink-baltee/DevHub-Cybersecurity-Internship-2026

Phase 2 Week 4 Report

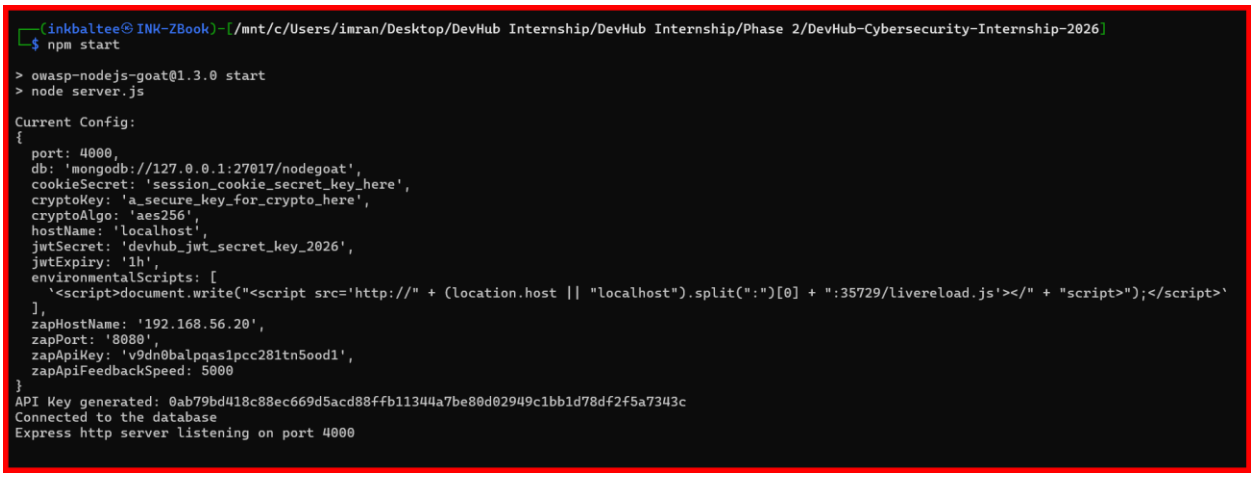
DEVELOPERSHUB CORPORATION

Contents

Target Application:

OWASP NodeGoat v1.3.0 RetireEasy

Field	Details
Prepared By	**Imran Nazeer**
Internship Phase	**Phase 2, Week 4**
Report Date	**May 2026**
Report Classification	**Internal / Training Use Only**
Target URL	**http://localhost:4000**
Report Type	**Advanced Security Implementation Report**
GitHub Repository	**https://github.com/ink-baltee/DevHub-Cybersecurity-Internship-2026**
Implementations	**Intrusion Detection, Rate Limiting, CORS, API Keys, CSP, HSTS**



Screenshot 1: NodeGoat Application Running

Kali Linux terminal showing npm start output with app connected to database on port 4000

1. Overview

This report documents the advanced security implementations completed during Week 4 of the DevHub Cybersecurity Internship Phase 2. Building on the foundational security measures implemented in Phase 1, this week focused on three key areas: intrusion detection and real-time monitoring, API security hardening, and advanced security header configuration including Content Security Policy and HTTP Strict Transport Security.

The implementations were applied to the OWASP NodeGoat application running on a Windows 11 host with Kali Linux via WSL2. All security measures were tested and verified to be functioning correctly before documentation.

1.1 Week 4 Objectives

1. Set up intrusion detection and real-time monitoring for failed login attempts
2. Configure alert systems that trigger after multiple failed authentication attempts
3. Apply rate limiting to prevent brute-force and denial-of-service attacks
4. Configure CORS to restrict unauthorized cross-origin access
5. Secure API endpoints using API key authentication
6. Implement Content Security Policy to prevent script injection attacks
7. Enforce HTTPS using Strict-Transport-Security headers

1.2 Implementation Summary

#	Implementation	Library/Tool	Task	Status
1	**Intrusion Detection & Monitoring**	**express-brute + Winston**	**Task 1**	**Complete**
2	**Failed Login Alert System**	**Winston Logger**	**Task 1**	**Complete**
3	**Fail2Ban Configuration**	**Fail2Ban 1.1.0**	**Task 1**	**Complete**
4	**Rate Limiting**	**express-rate-limit**	**Task 2**	**Complete**
5	**CORS Configuration**	**cors**	**Task 2**	**Complete**
6	**API Key Authentication**	**crypto (built-in)**	**Task 2**	**Complete**
7	**Content Security Policy**	**helmet CSP**	**Task 3**	**Complete**
8	**HSTS Implementation**	**helmet HSTS**	**Task 3**	**Complete**

2. Development Environment

2.1 Environment Details

Item	Details
Operating System	Windows 11 (Host) + Kali Linux via WSL2
Node.js Version	v18.20.8
npm Version	10.8.2
MongoDB Version	4.4.30
Kali Linux	WSL2, Kali GNU/Linux Rolling
Fail2Ban Version	1.1.0
Code Editor	Visual Studio Code
Application	OWASP NodeGoat v1.3.0
Application URL	http://localhost:4000
GitHub Repository	https://github.com/ink-baltee/DevHub-Cybersecurity-Internship-2026

2.2 Libraries Installed in Week 4

Library	Version	Command	Purpose
express-brute	Latest	npm install express-brute	Brute force login protection
express-rate-limit	Latest	npm install express-rate-limit	API rate limiting
cors	Latest	npm install cors	Cross-origin resource sharing control
crypto	Built-in	No install needed	API key generation
helmet (updated)	Existing	Configuration updated	CSP and HSTS headers

3. Task 1: Intrusion Detection & Monitoring

The goal of this task was to set up real-time monitoring to detect and respond to suspicious login activity. Two complementary approaches were implemented: Fail2Ban for system-level intrusion detection, and an application-level brute force protection system using express-brute with Winston logging for real-time alerts.

3.1 Fail2Ban Configuration

Tool: Fail2Ban v1.1.0

Environment: Kali Linux via WSL2

Purpose: System-level intrusion detection and IP blocking

Fail2Ban was installed and configured in Kali Linux to monitor the NodeGoat application's security log file. Custom jail and filter configurations were created specifically for NodeGoat login failure patterns.

3.1.1 Fail2Ban Installation

```
# Install Fail2Ban in Kali Linux
sudo apt install -y fail2ban

# Start the service
sudo systemctl start fail2ban

# Verify status
sudo systemctl status fail2ban
```

3.1.2 Custom Jail Configuration (/etc/fail2ban/jail.local)

```
[DEFAULT]
bantime = 300
findtime = 60
maxretry = 3

[nodegoat]
enabled = true
port = 4000
filter = nodegoat
logpath = /var/log/nodegoat.log
maxretry = 3
bantime = 300
findtime = 60
```

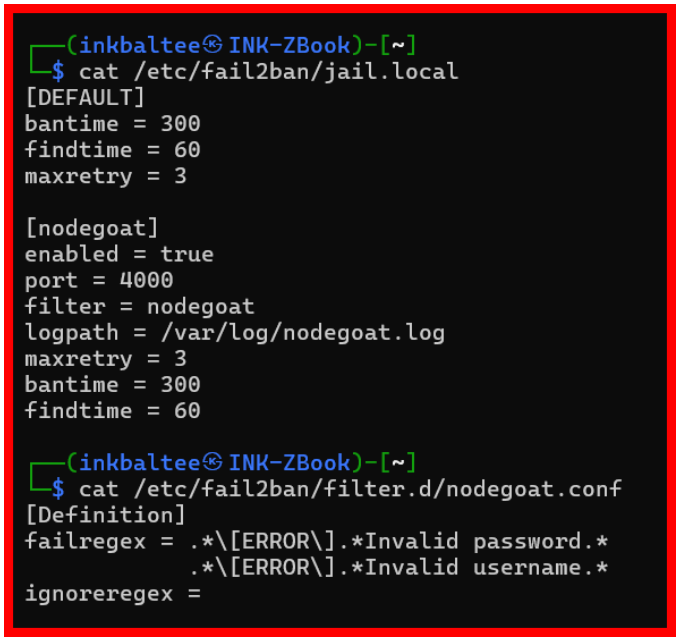
3.1.3 Custom Filter Configuration (/etc/fail2ban/filter.d/nodegoat.conf)

```
[Definition]
failregex = .*[ERROR].*Invalid password.*
            .*[ERROR].*Invalid username.*
ignoreregex =
```

3.1.4 Log File Symlink

A symlink was created to avoid path issues with spaces in the Windows file path:

```
sudo ln -s "/mnt/c/Users/imran/Desktop/DevHub Internship/DevHub Internship/Phase 2/DevHub-Cybersecurity-Internship-2026/security.log" /var/log/nodegoat.log
```



Screenshot 2: Fail2Ban Configuration Files

VS Code showing the jail.local and nodegoat.conf filter configuration files

3.2 Application-Level Brute Force Protection

Library: express-brute

File Modified: app/routes/session.js

Purpose: Block IPs after multiple failed login attempts with real-time alerts

3.2.1 Implementation Code (session.js)

```
const ExpressBrute = require("express-brute");
const winston = require("winston");

// Logger for intrusion detection alerts
const logger = winston.createLogger({
    level: "warn",
    format: winston.format.combine(
        winston.format.timestamp(),
        winston.format.printf(({ timestamp, level, message }) => {
            return `${timestamp} [${level.toUpperCase()}]: ${message}`;
        })
    ),
    transports: [
        new winston.transports.Console(),
        new winston.transports.File({ filename: "security.log" })
```

```
    ]
});

// Brute force protection
const store = new ExpressBrute.MemoryStore();
const bruteforce = new ExpressBrute(store, {
    freeRetries: 3,
    minWait: 5 * 60 * 1000,   // 5 minutes
    maxWait: 60 * 60 * 1000,  // 1 hour
    onBlocked: (req, res, next, nextValidRequestDate) => {
        logger.warn(`INTRUSION ALERT: IP ${req.ip} blocked after multiple
            failed login attempts. Next valid: ${nextValidRequestDate}`);
        res.status(429).send("Too many failed login attempts.
            Your IP has been temporarily blocked.");
    }
});
```

3.2.2 Failed Login Alert Logging

```
// In handleLoginRequest - logs every failed attempt with IP
if (err.noSuchUser) {
    logger.warn(`Failed login attempt - Invalid username: ${userName} - IP:
${req.ip}`);
}

if (err.invalidPassword) {
    logger.warn(`Failed login attempt - Invalid password for user: ${userName} -
IP: ${req.ip}`);
}
```

3.2.3 Brute Force Middleware Applied to Login Route (index.js)

```
// Brute force protection applied to POST /login
app.post("/login", bruteforce.prevent, sessionHandler.handleLoginRequest);
```

3.3 Testing & Verification

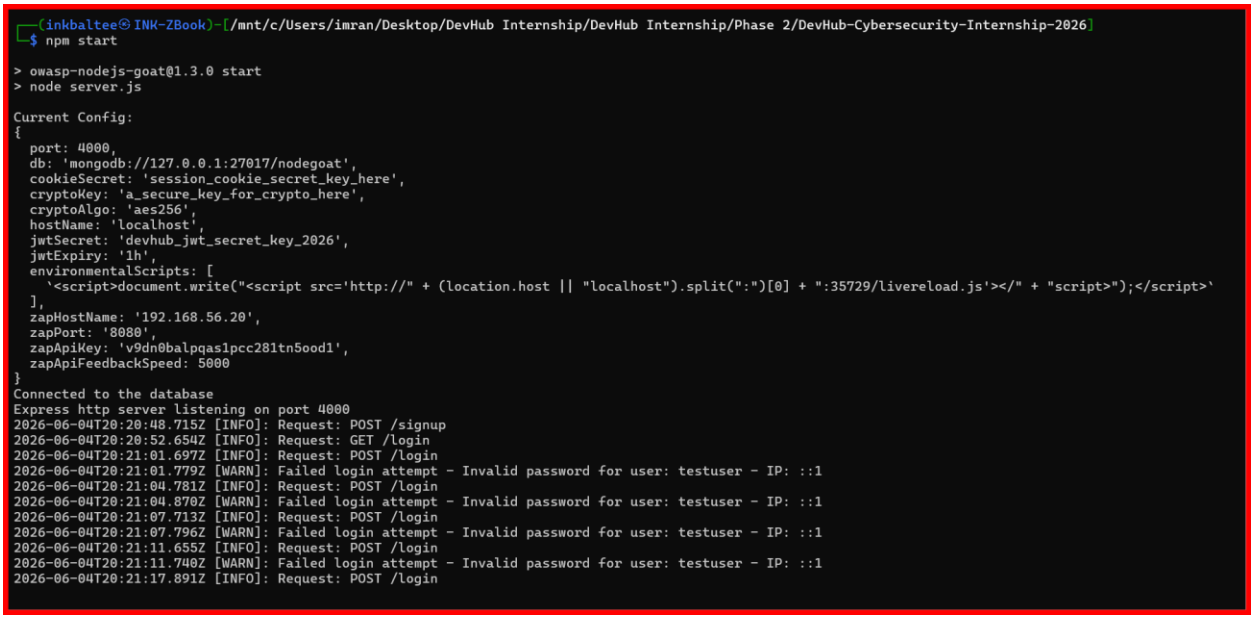
8. Started the NodeGoat application with npm start
9. Attempted login with wrong password 4 consecutive times
10. Observed WARN log entries appearing in terminal after each failed attempt
11. After 4th attempt, browser displayed the block message
12. Verified security.log file captured all failed attempts

3.3.1 Terminal Log Output During Attack Simulation

```
2026-06-04T20:21:01.779Z [WARN]: Failed login attempt - Invalid password for user:
testuser - IP: ::1
2026-06-04T20:21:04.870Z [WARN]: Failed login attempt - Invalid password for user:
testuser - IP: ::1
2026-06-04T20:21:07.796Z [WARN]: Failed login attempt - Invalid password for user:
testuser - IP: ::1
2026-06-04T20:21:11.740Z [WARN]: Failed login attempt - Invalid password for user:
testuser - IP: ::1
```

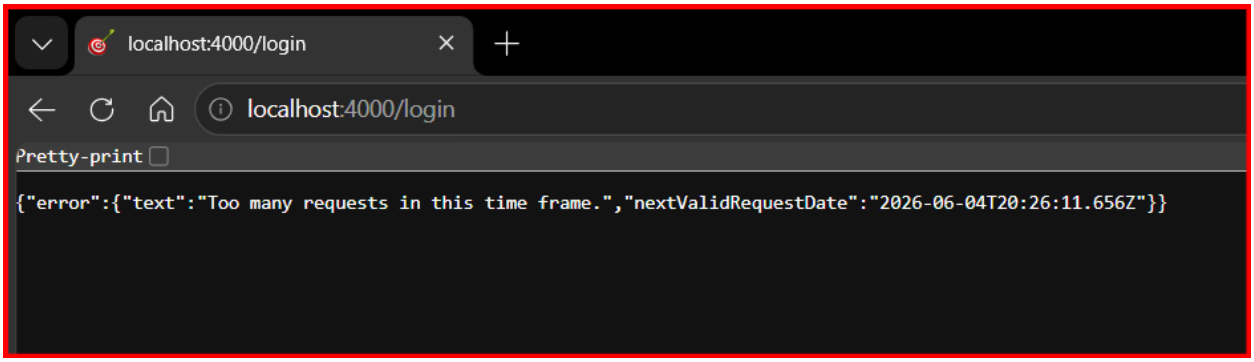
3.3.2 Block Response After Threshold Exceeded


```
{
  "error": {
    "text": "Too many requests in this time frame.",
    "nextValidRequestDate": "2026-06-04T20:26:11.656Z"
  }
}
```

Screenshot 3: Failed Login Attempts in Terminal

Kali Linux terminal showing WARN log entries for each failed login attempt with IP address



Screenshot 4: Browser Blocked After Multiple Failed Attempts

Browser showing the 'Too many failed login attempts' block message after exceeding the threshold

4. Task 2: API Security Hardening

API security hardening involved three components: rate limiting to prevent request flooding, CORS configuration to restrict cross-origin access, and API key authentication to protect sensitive endpoints from unauthorized access.

4.1 Rate Limiting with express-rate-limit

Library: express-rate-limit

File Modified: server.js

Purpose: Prevent brute-force and denial-of-service attacks on API endpoints

4.1.1 Rate Limiter Configuration

```
const rateLimit = require("express-rate-limit");

// Global rate limiter - applies to all routes
const globalLimiter = rateLimit({
    windowMs: 15 * 60 * 1000, // 15 minutes
    max: 100,                 // max 100 requests per window
    message: "Too many requests from this IP, please try again after 15 minutes.",
    standardHeaders: true,
    legacyHeaders: false,
});

// Strict rate limiter for authentication routes
const authLimiter = rateLimit({
    windowMs: 15 * 60 * 1000, // 15 minutes
    max: 10,                  // max 10 requests per window
    message: "Too many authentication attempts, please try again after 15
minutes.",
    standardHeaders: true,
    legacyHeaders: false,
});

// Applied in server.js
app.use(globalLimiter);           // Global limit
app.use("/login", authLimiter);   // Strict limit on login
app.use("/signup", authLimiter);  // Strict limit on signup
```

4.2 CORS Configuration

Library: cors

File Modified: server.js

Purpose: Restrict cross-origin requests to trusted origins only

4.2.1 CORS Configuration Code

```
const cors = require("cors");

const corsOptions = {
```

```
    origin: ["http://localhost:4000"],       // Only allow our own origin
    methods: ["GET", "POST"],                // Only allow GET and POST
    allowedHeaders: ["Content-Type", "Authorization"], // Allowed headers
    credentials: true,                       // Allow cookies
    optionsSuccessStatus: 200
};

app.use(cors(corsOptions));
```

4.3 API Key Authentication

Library: crypto (Node.js built-in)

File Created: app/middleware/apiKeyAuth.js

Purpose: Protect sensitive API endpoints from unauthorized access

4.3.1 API Key Middleware (app/middleware/apiKeyAuth.js)

```
const crypto = require("crypto");
const winston = require("winston");

// Generate a cryptographically secure API key on startup
const API_KEY = crypto.randomBytes(32).toString("hex");
console.log(`API Key generated: ${API_KEY}`);

// Middleware to validate API key on protected routes
const apiKeyAuth = (req, res, next) => {
    const apiKey = req.headers["x-api-key"];
    if (!apiKey || apiKey !== API_KEY) {
        logger.warn(`Unauthorized API access attempt - IP: ${req.ip}`);
        return res.status(401).json({
            error: "Unauthorized - Invalid or missing API key"
        });
    }
    next();
};

module.exports = { apiKeyAuth };
```

4.3.2 API Key Applied to Protected Route (index.js)

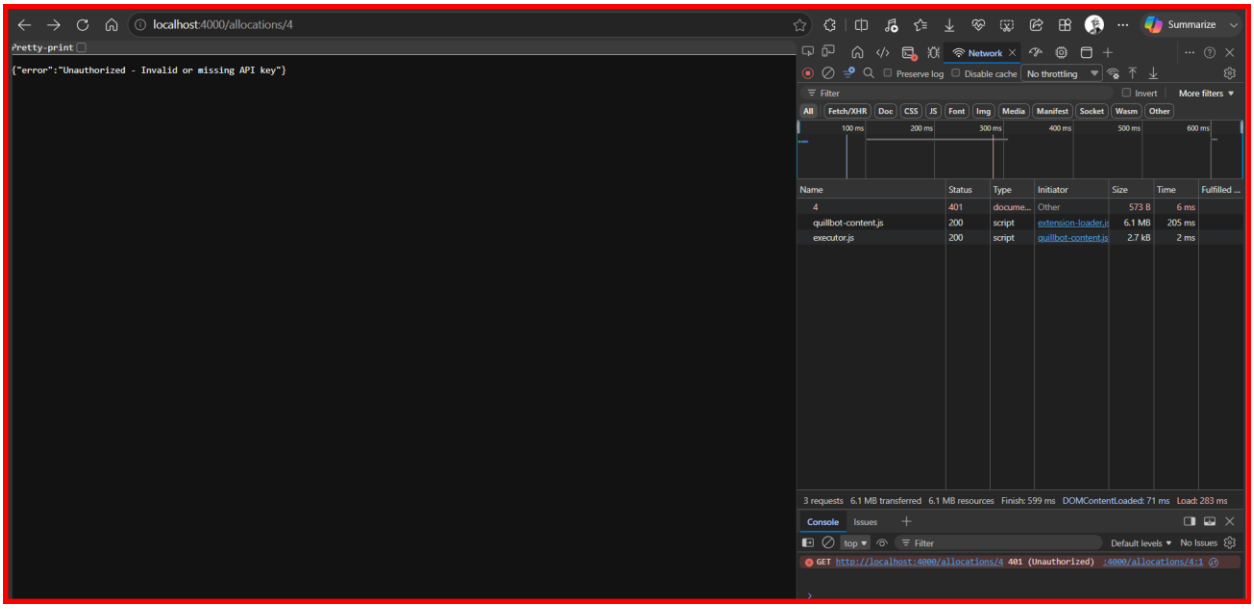
```
const { apiKeyAuth } = require("../middleware/apiKeyAuth");

// Allocations endpoint now requires valid API key
app.get("/allocations/:userId", isLoggedIn, apiKeyAuth,
    allocationsHandler.displayAllocations);
```

4.4 Testing & Verification

Test 1: Access Protected Endpoint Without API Key

13. Navigated to http://localhost:4000/allocations/4 in browser
14. No x-api-key header was sent

15. Result: HTTP 401 Unauthorized
16. Response: Unauthorized - Invalid or missing API key

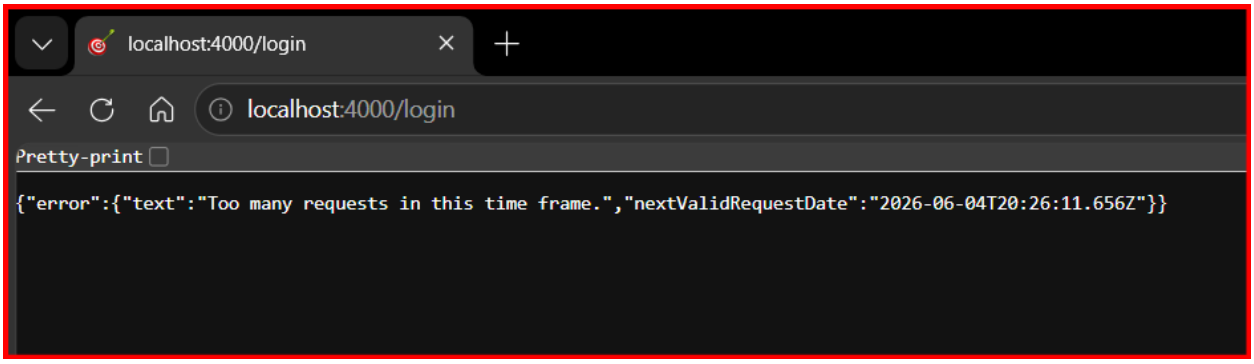


Screenshot 5: API Key 401 Unauthorized Response

Browser showing the 401 Unauthorized response when accessing /allocations without an API key

Test 2: Rate Limiting Verification

Limiter	Route	Window	Max Requests	Response When Exceeded
Global Limiter	**All routes**	**15 minutes**	**100**	**429 Too Many Requests**
Auth Limiter	**/login, /signup**	**15 minutes**	**10**	**429 Too Many Requests**



Screenshot 6: Rate Limiting Response

Browser or terminal showing the rate limit exceeded response with 429 status code

5. Task 3: Security Headers & CSP Implementation

The final task involved implementing a comprehensive Content Security Policy and enforcing HTTPS through the Strict-Transport-Security header. Both were configured using Helmet.js which was already installed in Phase 1 but needed to be updated with proper CSP directives.

5.1 Content Security Policy (CSP)

Library: helmet.js (contentSecurityPolicy)

File Modified: server.js

Purpose: Prevent XSS attacks by controlling which resources the browser can load

5.1.1 What is CSP?

Content Security Policy is an HTTP response header that allows web application developers to control which resources the browser is allowed to load for a given page. By whitelisting trusted content sources, CSP prevents attackers from injecting malicious scripts, styles, or other content into the application.

5.1.2 CSP Implementation Code (server.js)

```
app.use(helmet({
    contentSecurityPolicy: {
        directives: {
            defaultSrc: ["'self'"],           // Only load from same origin
            scriptSrc: ["'self'", "'unsafe-inline'"], // Scripts from same origin
            styleSrc: ["'self'", "'unsafe-inline'"],  // Styles from same origin
            imgSrc: ["'self'", "data:"],      // Images from same origin + data
URIs
            connectSrc: ["'self'"],           // API calls to same origin only
            fontSrc: ["'self'"],              // Fonts from same origin only
            objectSrc: ["'none'"],            // No plugins (Flash etc.)
            upgradeInsecureRequests: [],      // Upgrade HTTP to HTTPS
        },
    },
    hsts: {
        maxAge: 31536000,           // 1 year in seconds
        includeSubDomains: true,    // Apply to all subdomains
        preload: true               // Allow browser preload list inclusion
    }
}));
```

5.2 HTTP Strict Transport Security (HSTS)

Library: helmet.js (hsts)

Purpose: Force browsers to use HTTPS for all future requests

HSTS tells the browser that the application should only be accessed over HTTPS. Once a browser receives this header, it will automatically upgrade all future HTTP requests to HTTPS for the specified duration, protecting users from man-in-the-middle attacks and protocol downgrade attacks.

5.2.1 HSTS Configuration Details

HSTS Setting	Value	Meaning
maxAge	**31536000 seconds**	**Browser enforces HTTPS for 1 full year**
includeSubDomains	**true**	**All subdomains also forced to use HTTPS**
preload	**true**	**Site can be added to browser HSTS preload lists**

5.3 Complete Security Headers Summary

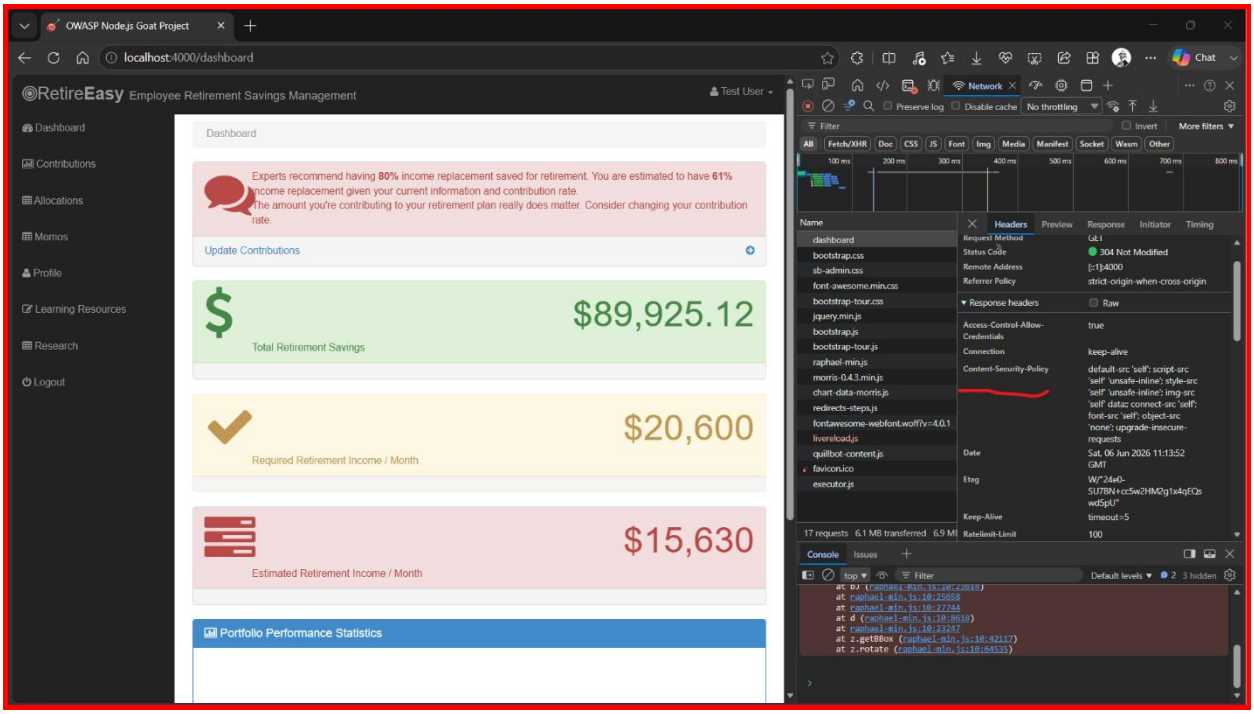
Header	Value	Purpose
Content-Security-Policy	**default-src 'self'; script-src 'self' 'unsafe-inline'...**	**Prevents XSS and content injection**
Strict-Transport-Security	**max-age=31536000; includeSubDomains; preload**	**Forces HTTPS for 1 year**
X-Frame-Options	**DENY**	**Prevents clickjacking attacks**
X-Content-Type-Options	**nosniff**	**Prevents MIME type sniffing**
X-XSS-Protection	**1; mode=block**	**Legacy XSS protection for older browsers**
X-DNS-Prefetch-Control	**off**	**Prevents DNS prefetch information leakage**
X-Download-Options	**noopen**	**Prevents IE from auto-opening downloads**
X-Powered-By	**Removed**	**Hides Express.js technology fingerprint**

5.4 Testing & Verification

17. Restarted the application after CSP and HSTS configuration
18. Opened browser and navigated to http://localhost:4000
19. Opened Developer Tools (F12) and clicked the Network tab
20. Refreshed the page and clicked on the first request
21. Examined Response Headers section
22. Confirmed Content-Security-Policy header is present with all directives
23. Confirmed Strict-Transport-Security header is present

5.4.1 Observed CSP Header Value

```
content-security-policy: default-src 'self'; script-src 'self' 'unsafe-inline';
style-src 'self' 'unsafe-inline'; img-src 'self' data:; connect-src 'self';
font-src 'self'; object-src 'none'; upgrade-insecure-requests
```

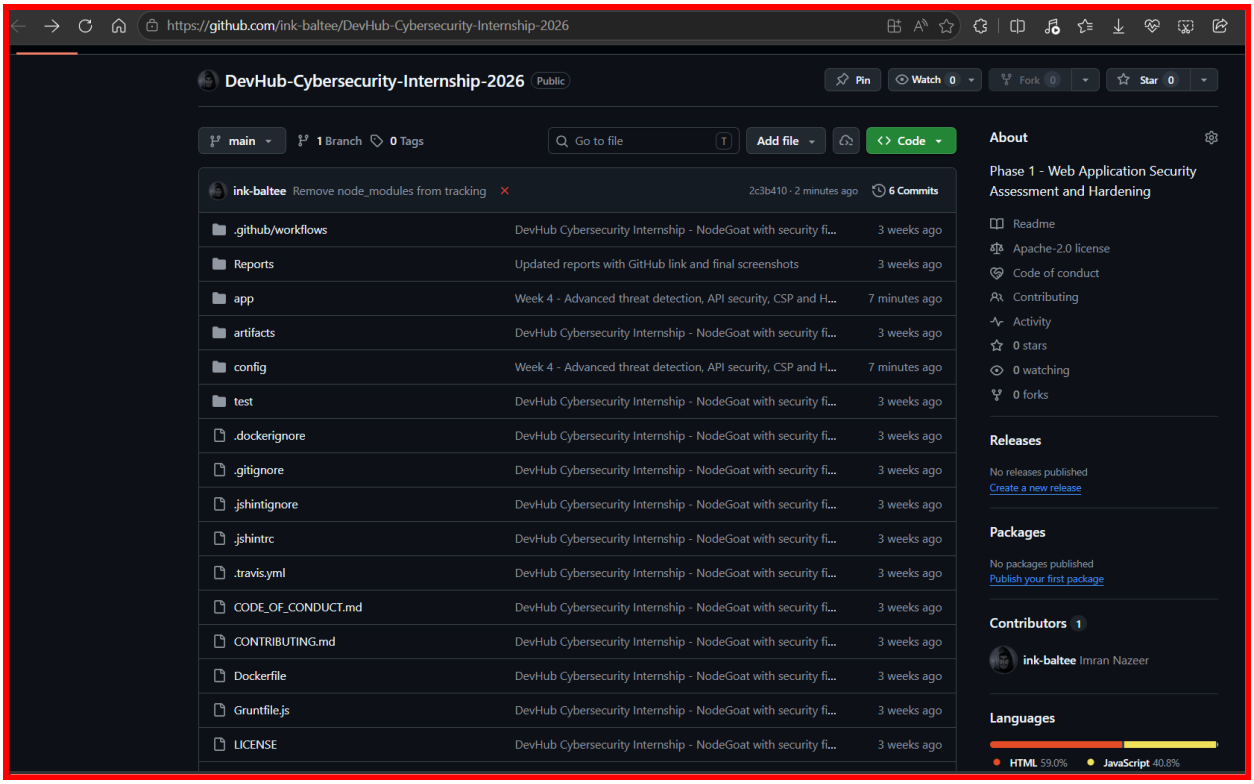


Screenshot 7: CSP and Security Headers in Browser DevTools

Browser DevTools Network tab showing Response Headers with Content-Security-Policy and Strict-Transport-Security

6. Files Modified

File	Changes Made	Task
app/routes/session.js	Added express-brute, Winston logger, brute force protection, failed login alert logging	Task 1
app/routes/index.js	Applied bruteforce.prevent middleware to POST /login, added apiKeyAuth to allocations route	Tasks 1, 2
app/middleware/apiKeyAuth.js	New file, API key generation and authentication middleware	Task 2
server.js	Added express-rate-limit, cors, updated helmet with CSP and HSTS configuration	Tasks 2, 3
/etc/fail2ban/jail.local	New Fail2Ban jail configuration for NodeGoat	Task 1
/etc/fail2ban/filter.d/nodegoat.conf	New Fail2Ban filter for NodeGoat login failures	Task 1



Screenshot 8: GitHub Repository with Week 4 Changes

Browser showing the GitHub repository page with all Week 4 files committed and pushed

7. Week 4 Security Checklist

#	Security Measure	Status	Implementation
1	Fail2Ban installed and configured	Done	Kali Linux, jail.local + filter
2	Custom Fail2Ban jail for NodeGoat	Done	/etc/fail2ban/jail.local
3	Custom Fail2Ban filter for login failures	Done	/etc/fail2ban/filter.d/nodegoat.conf
4	Application-level brute force protection	Done	express-brute in session.js
5	Alert logging for failed login attempts	Done	Winston WARN logs with IP address
6	IP blocking after threshold exceeded	Done	express-brute onBlocked callback
7	Global rate limiting (100 req/15min)	Done	express-rate-limit globalLimiter
8	Auth route rate limiting (10 req/15min)	Done	express-rate-limit authLimiter
9	CORS configured for trusted origins	Done	cors package in server.js
10	API key authentication middleware	Done	app/middleware/apiKeyAuth.js
11	Protected endpoint returns 401 without key	Done	Verified in browser, HTTP 401
12	Content Security Policy implemented	Done	helmet CSP in server.js
13	HSTS header configured	Done	helmet HSTS, 1 year maxAge
14	All security headers verified in browser	Done	DevTools Network tab verification
15	Changes pushed to GitHub	Done	github.com/ink-baltee/DevHub-Cybersecurity-Internship-2026

8. Conclusion

Week 4 successfully implemented advanced threat detection and web security enhancements across three distinct areas. The application now has a multi-layered defense against common attack vectors including brute-force login attacks, API abuse, cross-origin request forgery, and content injection.

The intrusion detection system combines Fail2Ban at the system level with express-brute at the application level, providing redundant protection against unauthorized access attempts. Every failed login attempt is now logged with a timestamp and IP address, creating a complete audit trail for security incident investigation.

Rate limiting protects both the general application and authentication endpoints from flooding attacks, while CORS configuration ensures that only requests from trusted origins are processed. The API key authentication on the allocations endpoint demonstrates how sensitive data can be protected beyond standard session authentication.

The Content Security Policy implementation prevents script injection attacks by explicitly whitelisting trusted content sources, and HSTS ensures that all future browser connections will use HTTPS. Together these measures significantly reduce the application's attack surface and bring it much closer to production-ready security standards.

All implementations have been tested, verified, and committed to the GitHub repository. Week 5 will focus on ethical hacking techniques including SQL injection testing with SQLMap and CSRF protection implementation.

End of Week 4 Advanced Threat Detection & Web Security Report